COMPTE RENDU DE TP

# Installation et validation d’Apache Spark

## TP1 • Big Data • Apache Spark 4.2.0

Date de réalisation : 20 octobre 2026
Auteur : nom non communiqué
Environnement : Windows, exécution locale

## Objectif et méthode

Installer un JDK compatible, vérifier l’archive Spark, configurer les processus puis valider les interfaces Scala et Python, le calcul de Pi et l’interface Web.

Les résultats de ce document proviennent des exécutions effectuées sur cette machine. Le ZIP contient les journaux bruts, les captures séparées, les scripts et la configuration utilisée.

### Nature des captures

Les preuves de console sont des captures, dans un navigateur, de journaux réels mis en page pour leur lisibilité. Elles ne représentent pas une fenêtre PowerShell. Les figures Web sont des captures directes de l’interface Spark en fonctionnement. Les illustrations du sujet ne sont pas réutilisées.

# 1. Environnement et installation

Un JDK 17 portable est extrait dans le dossier de travail. Java 11, déjà présent dans le PATH de la machine, est remplacé uniquement dans les processus du TP. Version vérifiée : 17.0.20.1.

**TP1 Spark - 01_java.txt**

Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
$ C:\Users\lenovo\Documents\Codex\2026-10-20\fa-2\work\java\jdk-17.0.20.1+1\bin\java.exe -version

openjdk version "17.0.20.1" 2026-08-18
OpenJDK Runtime Environment Temurin-17.0.20.1+1 (build 17.0.20.1+1)
OpenJDK 64-Bit Server VM Temurin-17.0.20.1+1 (build 17.0.20.1+1, mixed mode, sharing)

Code de sortie : 0
```

*Figure 1 - Sortie réelle de java -version.*

L'archive spark-4.2.0-bin-hadoop3.tgz est téléchargée depuis le miroir officiel Apache. Sa somme SHA-512 est comparée au fichier .sha512 publié par Apache avant extraction. Une correspondance exacte est exigée par le script.

**TP1 Spark - 02_integrite.txt**

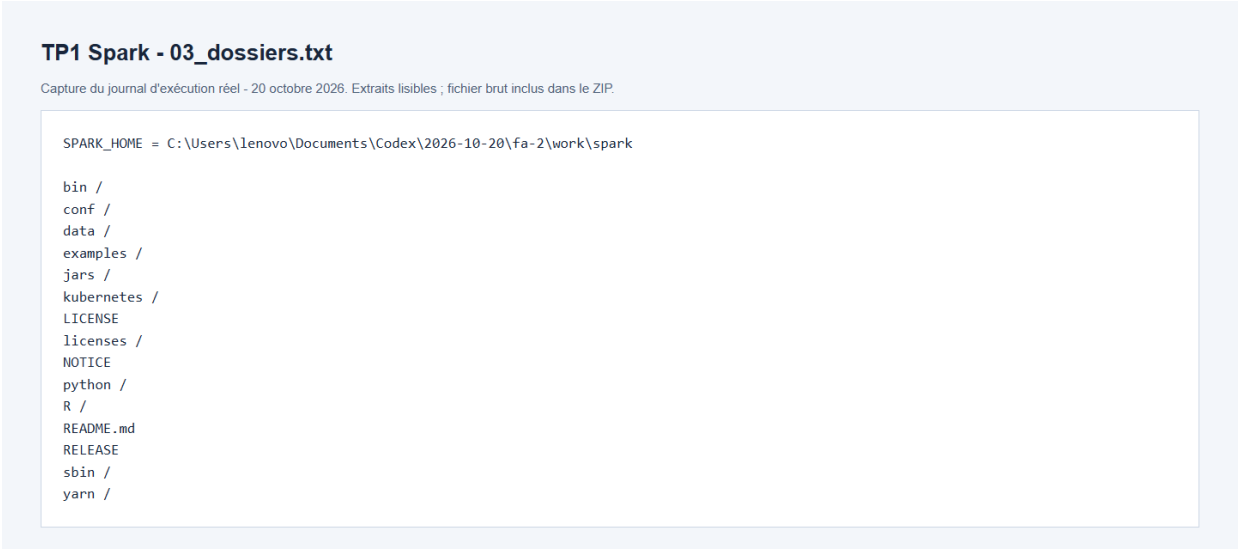
Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
Archive : spark.tgz
Source : https://dlcdn.apache.org/spark/spark-4.2.0/spark-4.2.0-bin-hadoop3.tgz
SHA-512 calcule :
3a6559e8546ff387db8fe7a04b8fe4008853b467b972c2e343df8e14a81450534777719fc5d4998dd96519a86fdf9de140cee753c61e2e94db044d5f9555ddc4
SHA-512 officiel :
3a6559e8546ff387db8fe7a04b8fe4008853b467b972c2e343df8e14a81450534777719fc5d4998dd96519a86fdf9de140cee753c61e2e94db044d5f9555ddc4
Correspondance : True
```

*Figure 2 - Comparaison de la somme de contrôle SHA-512.*

# 2. Dossiers et variables d’environnement

Spark est extrait dans le sous-dossier work/spark du dossier de cette conversation, au lieu de C:\spark. Les dossiers bin, conf, jars, examples et python sont contrôlés après extraction.



**TP1 Spark - 03_dossiers.txt**

Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
SPARK_HOME = C:\Users\lenovo\Documents\Codex\2026-10-20\fa-2\work\spark

bin /
conf /
data /
examples /
jars /
kubernetes /
LICENSE
licenses /
NOTICE
python /
R /
README.md
RELEASE
sbin /
yarn /
```

*Figure 3 - Contenu réel du répertoire d’installation.*

JAVA_HOME désigne le JDK ; SPARK_HOME désigne la racine de Spark. Leurs dossiers bin sont placés en tête du PATH des processus lancés. Ces variables sont locales aux tests et ne sont pas enregistrées dans les paramètres utilisateur Windows. Le script fourni permet de les rétablir dans une session PowerShell.



**TP1 Spark - 04_environnement.txt**

Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
Variables appliquees aux processus de ce TP (non persistantes) :

JAVA_HOME = C:\Users\lenovo\Documents\Codex\2026-10-20\fa-2\work\java\jdk-17.0.20.1+1
SPARK_HOME = C:\Users\lenovo\Documents\Codex\2026-10-20\fa-2\work\spark
PYSPARK_PYTHON = C:\Users\lenovo\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe

PATH : JAVA_HOME\bin et SPARK_HOME\bin sont ajoutes en tete.
```

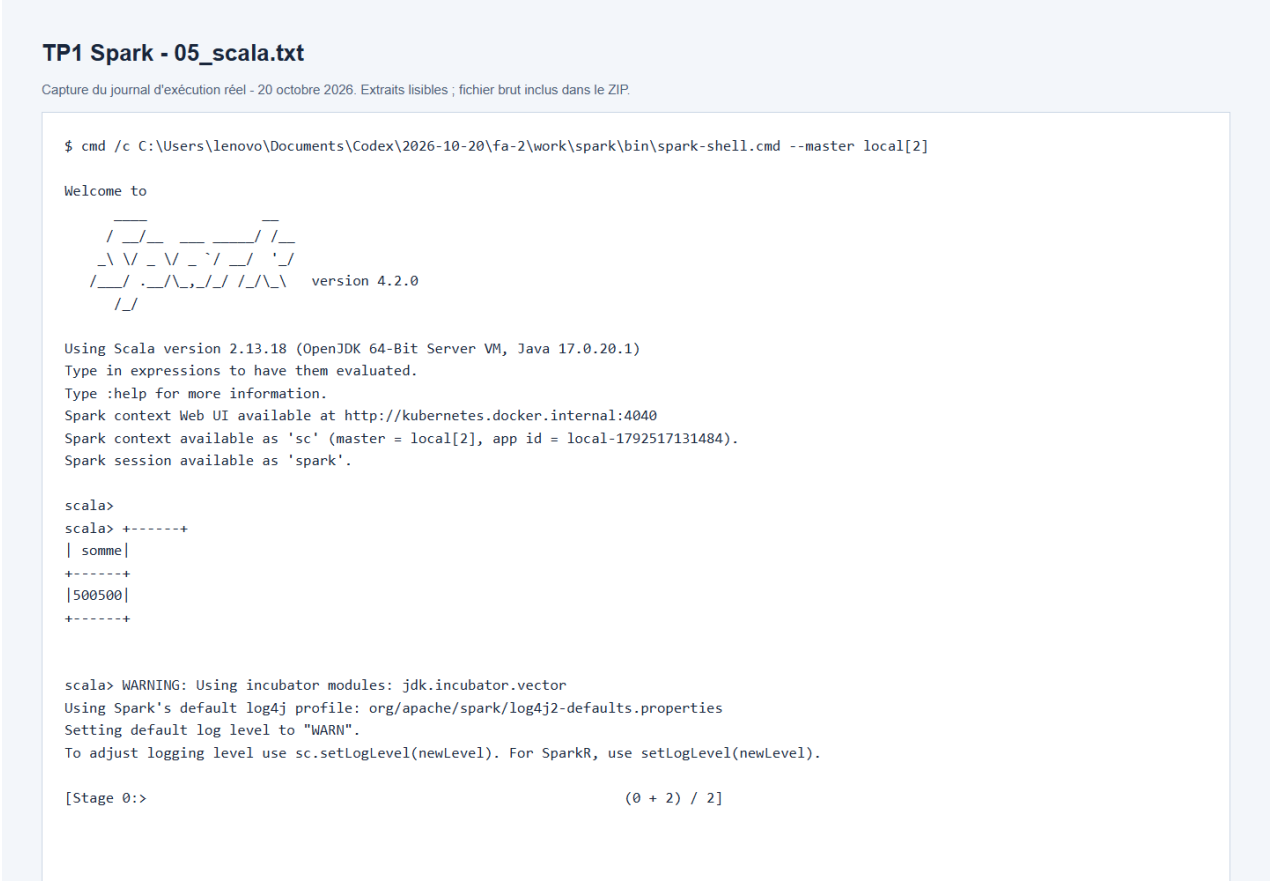
*Figure 4 - Valeurs appliquées aux processus du TP.*

# 3. Spark Shell : calcul en Scala

Le shell est lancé avec --master local[2]. Cette option autorise deux threads de travail sur la machine locale. Le calcul porte sur les entiers de 1 à 1000 inclus : la borne supérieure de range est exclusive.

```
spark.range(1, 1001).selectExpr("sum(id) AS somme").show()
```

La commande du sujet, spark.range(1, 1001).sum(), est remplacée par cette agrégation explicite sur la colonne id. La vérification théorique est 1000 × 1001 / 2 = 500500.



**TP1 Spark - 05_scala.txt**

Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
$ cmd /c C:\Users\lenovo\Documents\Codex\2026-10-20\fa-2\work\spark\bin\spark-shell.cmd --master local[2]

Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.2.0
      /_/

Using Scala version 2.13.18 (OpenJDK 64-Bit Server VM, Java 17.0.20.1)
Type in expressions to have them evaluated.
Type :help for more information.
Spark context Web UI available at http://kubernetes.docker.internal:4040
Spark context available as 'sc' (master = local[2], app id = local-1792517131484).
Spark session available as 'spark'.

scala>
scala> +------+
| somme|
+------+
|500500|
+------+


scala> WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).

[Stage 0:>                                                          (0 + 2) / 2]
```

*Figure 5 - Démarrage de Spark Shell et résultat de l'agrégation.*

# 4. Exemple SparkPi et API Python

Le lancement initial par run-example.cmd SparkPi 10 échoue : Hadoop réclame winutils.exe lors de la copie du JAR. Le même exemple intégré est alors exécuté avec spark-class.cmd org.apache.spark.examples.SparkPi 10, en ajoutant examples/jars/* à SPARK_DIST_CLASSPATH et en fixant spark.master=local[2] via JAVA_TOOL_OPTIONS. Cette exécution évite la copie du JAR. Résultat observé : **3.1402751402751403**. Cette estimation aléatoire varie entre les lancements.

**TP1 Spark - 06_sparkpi.txt**

Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
$ spark-class.cmd org.apache.spark.examples.SparkPi 10 (SPARK_DIST_CLASSPATH=examples/jars/* ; JAVA_TOOL_OPTIONS=-Dspark.master=local[2] -
Dspark.ui.port=4041)
WARNING: Using incubator modules: jdk.incubator.vector
26/10/20 17:27:03 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and
hadoop.home.dir are unset. -see https://cwiki.apache.org/confluence/display/HADOOP2/WindowsProblems
26/10/20 17:27:04 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where
applicable
Pi is roughly 3.1402751402751403
Code de sortie : 0
```

*Figure 6 - Résultat réel de SparkPi.*

L'API PySpark est testée par spark-submit : range(5).show() affiche les valeurs 0 à 4 et une deuxième action calcule 500500. La commande pyspark est également validée avec les mêmes valeurs de 0 à 4. Pour alimenter cette session par une entrée redirigée, PYSPARK_DRIVER_PYTHON_OPTS=-i est nécessaire. Sa capture et son journal sont fournis dans le ZIP.

**TP1 Spark - 07_pyspark.txt**

Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
WARNING: Using incubator modules: jdk.incubator.vector
C:\Users\lenovo\Documents\Codex\2026-10-20\fa-2\work\spark\python\lib\pyspark.zip\pyspark\testing\utils.py:127: FutureWarning: PySpark
does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
SPARK_VERSION=4.2.0
+---+
| id|
+---+
|  0|
|  1|
|  2|
|  3|
|  4|
+---+

+------+
| somme|
+------+
|500500|
+------+

SHUFFLE_PARTITIONS=8
UI=http://kubernetes.docker.internal:4040
PRET_CAPTURE
```

*Figure 7 - Résultats du programme PySpark.*

# 5. Interface Web et configuration

L’application Python reste temporairement active pour consulter http://127.0.0.1:4040/jobs/. Les actions précédentes y apparaissent sous forme de jobs. L’interface cesse d’être disponible lorsque l’application s’arrête.

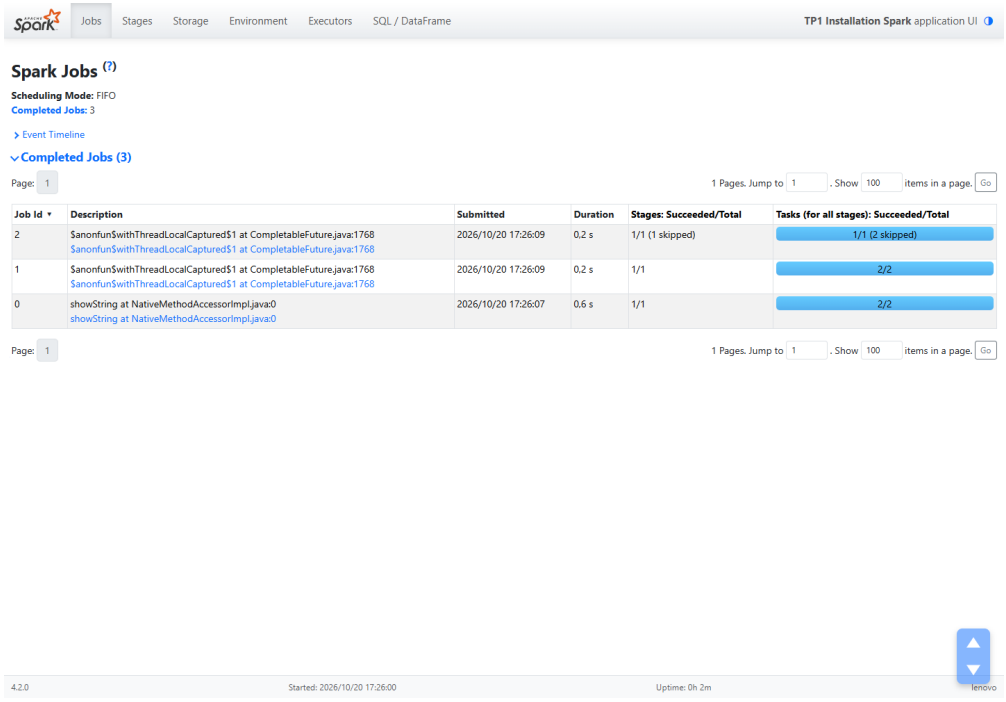



*Figure 8 - Capture directe de l’onglet Jobs de Spark.*

Le fichier conf/spark-defaults.conf fixe le mode local[*], 2g de mémoire driver, 8 partitions de shuffle et le port 4040. Le test utilise explicitement --master local[2], qui remplace le réglage local[*] du fichier. Le nombre de partitions est également lu depuis la session Python.

**TP1 Spark - 08_configuration.txt**

Capture du journal d'exécution réel - 20 octobre 2026. Extraits lisibles ; fichier brut inclus dans le ZIP.

```
spark.master                local[*]
spark.driver.memory         2g
spark.sql.shuffle.partitions  8
spark.ui.port               4040
```

*Figure 9 - Fichier de configuration utilisé.*

# 6. Bilan des vérifications

| Contrôle | Résultat observé |
|---|---|
| Java compatible | OK - JDK 17.0.20.1 |
| SHA-512 de l’archive | OK - correspondance exacte |
| SPARK_HOME | OK pour les processus ; chemin work/spark |
| Spark Shell 4.2.0 | OK |
| Somme Scala = 500500 | OK |
| SparkPi proche de 3,14 | 3.1402751402751403 |
| Interface Web sur 4040 | OK - capture directe effectuée |
| API PySpark et somme | OK |
| Commande interactive pyspark | OK - session démarrée et valeurs 0 à 4 |

## Écarts et précautions de lecture

L’installation est portable et les variables ne sont pas persistantes. Aucune capture de la boîte de dialogue Windows des variables d’environnement n’est fournie : les valeurs effectives des processus sont documentées. Les journaux peuvent contenir des avertissements Hadoop natifs ; leur présence ne remplace pas la vérification des résultats et des codes de sortie.

# 7. Réponses aux questions de compréhension

## 1. Quel est le rôle de SPARK_HOME ?

Cette variable indique le dossier racine de l'installation. Les scripts y retrouvent notamment les bibliothèques et les fichiers de configuration. Ajouter son sous-dossier bin au PATH permet d'appeler les commandes sans saisir leur chemin complet.

## 2. Que signifient local[2] et local[*] ?

local[2] exécute les tâches sur deux threads locaux. local[*] utilise autant de threads de travail que de processeurs logiques disponibles. Il s'agit toujours d'une exécution sur une seule machine.

## 3. Pourquoi Java ? Quelles versions sont compatibles ?

Le moteur Spark et son environnement Scala s'exécutent sur la JVM. PySpark pilote également ce moteur Java. La documentation Spark 4.2.0 indique Java 17, 21 et 25 ; les versions de Java 25 antérieures à 25.0.3 ont un support déprécié. Ce TP utilise Java 17.

## 4. À quoi sert l'interface Web du port 4040 ?

Elle permet de suivre les jobs, stages, tâches et exécuteurs, de consulter des métriques et d'examiner l'environnement de l'application. Elle aide à expliquer un traitement lent ou un échec. Elle appartient à une application active.

## 5. Différence entre spark-shell, pyspark et spark-submit ?

spark-shell ouvre une session interactive Scala. pyspark ouvre une session interactive Python reliée à Spark. spark-submit lance un programme enregistré, par exemple un script Python ou une application JVM, avec ses options d'exécution.

## Sources et fichiers de preuve

Sujet fourni : TP1_Installer_Spark.pdf, sections 3, 4, 7 et 8. Documentation officielle consultée le 20 octobre 2026 :

*https://spark.apache.org/downloads.html*

*https://spark.apache.org/docs/latest/*

*https://spark.apache.org/docs/latest/configuration.html*

*https://spark.apache.org/docs/latest/web-ui.html*

SUITE DU MÊME PROJET

# TP-0 : compléments
# TP-1 : WordCount

Apache Spark 4.2.0 • RDD, DataFrame et SQL
Réalisation : 20 octobre 2026 • Auteur : nom non communiqué

## Périmètre et reprise de l’existant

Le premier rapport d’installation est conservé une seule fois au début du PDF consolidé. Le présent complément traite les activités nouvelles du document TP0_TP1_Spark_4.2.0-1.pdf et des scripts fournis dans TP_Spark_sources_et_scripts.zip.

| Déjà acquis : réutilisé | Nouveau : réalisé dans cette suite |
|---|---|
| JDK 17, Spark 4.2.0 et Python 3.12 | Support natif Hadoop Windows pour les exports |
| Variables et configuration Spark existantes | Comparaison du parallélisme et somme des carrés |
| Shell Scala, PySpark, SparkPi, interface Web | WordCount RDD / DataFrame / SQL ; cinq exercices |

Les téléchargements et installations Java/Spark, SparkPi, range(5) et les sommes simples déjà validées ne sont pas refaits. Le script bonjour_spark.py et le deuxième exemple de Pi sont de même nature que les essais initiaux ; ils ne sont pas relancés. L’installation Ubuntu n’est pas reproduite sur ce poste Windows.

### Preuves

Les captures de résultats montrent dans un navigateur des extraits de journaux réellement produits, clairement identifiés. Les captures Spark UI sont prises directement dans l’application active. Les sources, mesures JSON et résultats CSV/texte sont inclus dans le ZIP.

# 8. Compléments du TP-0

Les chemins JAVA_HOME et SPARK_HOME restent ceux du projet existant. HADOOP_HOME est ajouté dans les processus de cette suite et pointe vers work/hadoop. Les binaires winutils.exe et hadoop.dll proviennent de hadoop-3.3.6/bin du dépôt cdarlint/winutils cité par le nouveau sujet. Leurs empreintes SHA-256 et leurs URL sont conservées ; ils sont aussi fournis dans outils/hadoop/bin pour la reproduction.

## Exercice 0.1 - variables

JAVA_HOME désigne le JDK, SPARK_HOME la racine Spark et HADOOP_HOME le dossier contenant bin/winutils.exe. Sans accès équivalent à Java, le moteur ne démarre pas. Sans chemin Spark correct, ses scripts ne retrouvent plus leurs bibliothèques. Sous Windows, l'absence du support Hadoop peut bloquer les accès et écritures de fichiers, comme l'avait montré le premier essai de SparkPi.

## Exercices 0.2 et 0.4 - nouvelles mesures




*Figure 10 - Parallélisme mesuré et calculs supplémentaires.*

La commande du nouveau calcul est `spark.range(1,101).selectExpr("sum(id*id)")`. La vérification mathématique donne 100 × 101 × 201 / 6 = 338350. local[2] autorise deux tâches simultanées ; local[*] utilise les processeurs logiques disponibles. defaultParallelism décrit le parallélisme par défaut, pas un relevé permanent de cœurs occupés.

## Exercice 0.3 - durée de vie de l'interface

Après l'arrêt de la session de la suite, la connexion au port 4040 est testée : connexion refusée après spark.stop(). L'interface appartient au driver actif ; elle disparaît à son arrêt. Ce test clôt les nouvelles observations de la Web UI, sans répéter le simple accès déjà validé.

# 9. Comprendre WordCount sur mini.txt

Le fichier UTF-8 fourni est réutilisé tel quel : trois lignes parlant de la mer, du capitaine et du Nautilus. Le découpage brut par espaces sert à l'exercice map/flatMap. Le comptage utilise ensuite des suites de lettres, la conversion en minuscules, puis le filtre de longueur > 2.

**Projet Spark - suite WordCount**

Capture de résultats réellement exécutés • 20 octobre 2026 • journaux bruts inclus

```
MINI.TXT - RDD / DataFrame / SQL

Lignes : 3
map(split).count() : 3
flatMap(split).count() : 20
Mots après normalisation : 19
Mots de plus de 2 lettres : 13
Mots uniques : 9

mer                             4
est                             2
belle                           1
calme                           1
capitaine                       1
nautilus                        1
plonge                          1
regarde                         1
sous                            1

Top 3 SQL : [['mer', 4], ['est', 2], ['belle', 1]]
Égalité RDD / DataFrame : True
Plus de 5 lettres : capitaine, nautilus, plonge, regarde
```

*Figure 11 - Résultats RDD et SQL, vérifiés avec DataFrame.*

## Exercices 1.1 et 1.2

map(split) produit une liste par ligne : 3 éléments. flatMap(split) concatène les listes : 20 éléments, dont le signe « ! » isolé. Après normalisation par expression régulière, il reste 19 mots, puis 13 occurrences avec le filtre > 2 et 9 mots distincts. Avec le filtre > 5, restent capitaine, nautilus, plonge et regarde.

## Chaîne de traitement

RDD : textFile → flatMap → filter → map(mot,1) → reduceByKey. DataFrame : read.text → lower/split/explode → filter → groupBy/count. SQL agrège la vue temporaire avec GROUP BY. Le tri ajoute le mot comme deuxième clé afin de stabiliser les ex æquo. Les neuf fréquences complètes sont identiques entre les deux API.

# 10. WordCount sur les cinq livres

Les cinq fichiers sont téléchargés depuis les URL du sujet. Les en-têtes et licences Gutenberg sont conservés, comme dans les scripts fournis : les chiffres décrivent donc les fichiers bruts, pas uniquement le texte littéraire. Le manifeste conserve les URL, tailles et empreintes des données exactes utilisées.

| Fichier | Ouvrage | Octets |
|---|---|---|
| pg17489.txt | Les Misérables, tome I | 710 496 |
| pg17989.txt | Le Comte de Monte-Cristo, tome I | 784 416 |
| pg13951.txt | Les Trois Mousquetaires | 1 450 561 |
| pg5097.txt | Vingt mille lieues sous les mers | 942 632 |
| pg4791.txt | Voyage au centre de la Terre | 461 113 |

Bilan : **496 765 occurrences** de mots de plus de deux lettres et **31 052 mots distincts**, sur 93 311 lignes. Le RDD d'entrée comporte 5 partitions observées ; leur nombre dépend des fichiers et des paramètres de lecture.

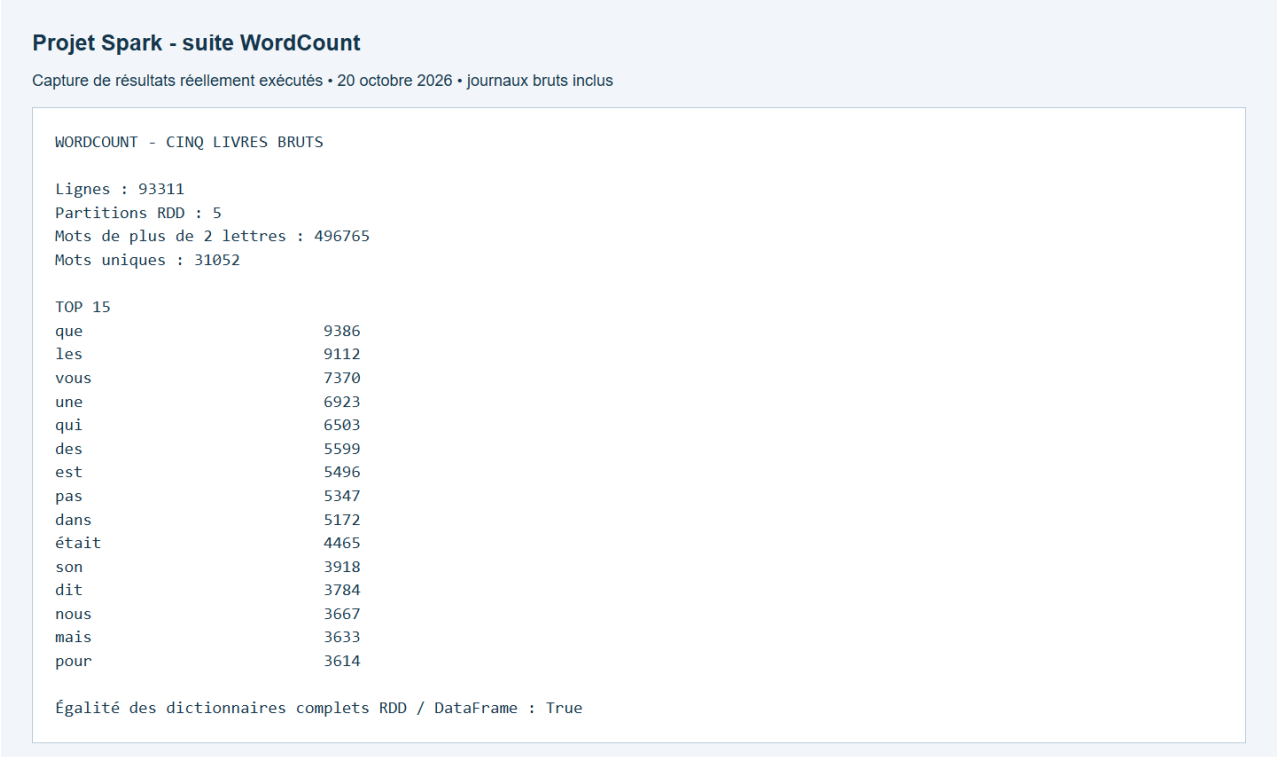



*Figure 12 - Top 15 réellement obtenu sur le corpus brut.*

# 11. Mots vides et comptage par livre

## Exercice 1.3 - exclusion des mots vides

La même liste est appliquée aux RDD et aux DataFrames : les, des, que, qui, dans, pour, pas, une, est. Les dictionnaires complets après filtrage sont identiques. Il reste 439 613 occurrences et 31 043 mots distincts. Cette liste limitée ne constitue pas une suppression complète des mots-outils français.

**Projet Spark - suite WordCount**

Capture de résultats réellement exécutés • 20 octobre 2026 • journaux bruts inclus

```
EXERCICES TP-1 : MOTS VIDES ET COMPTE PAR LIVRE

Mots vides retirés : dans, des, est, les, pas, pour, que, qui, une
Mots conservés : 439613
Égalité RDD / DataFrame : True

TOP 10 APRÈS EXCLUSION
vous                          7370
était                         4465
son                           3918
dit                           3784
nous                          3667
mais                          3633
sur                           3605
lui                           3496
avait                         3474
plus                          3146

COMPTAGE PAR LIVRE (mots > 2 lettres)
pg13951.txt                 165311
pg17489.txt                  82580
pg17989.txt                  91506
pg4791.txt                   51402
pg5097.txt                  105966
Métadonnée utilisée : _metadata.file_name
```

*Figure 13 - Classement après exclusion et comptes par livre.*

## Exercice 1.4 - métadonnée du fichier

La colonne d'origine est ajoutée avant le découpage des lignes, puis les mots sont regroupés par livre. Le filtre > 2 est conservé afin que la somme des cinq comptes égale le total WordCount. Méthode réellement utilisée : _metadata.file_name. Les résultats sont également fournis dans resultats/par_livre.csv.

# 12. Exécution paresseuse et Spark UI

## Exercice 1.5 - fichier absent

La création du RDD sur fichier_qui_n_existe_pas.txt donne : OK, aucune action exécutée. L'action count() déclenche ensuite une erreur de chemin absent. La lecture est différée jusqu'à une action ; l'erreur observée est attendue et capturée, elle ne marque pas un échec du WordCount.

## RDD et DataFrame : ce que montrent les preuves

Le graphe RDD contient un ShuffledRDD au regroupement par clé. Le plan DataFrame, enregistré en intégralité dans 19_plan_dataframe.txt, montre les opérateurs de lecture, découpage, filtrage, agrégation et échange. Des cache() évitent de recalculer les mêmes données pour les contrôles ; le plan final peut donc inclure InMemoryTableScan et des étapes adaptées par AQE. La configuration existante de huit partitions de shuffle est conservée.

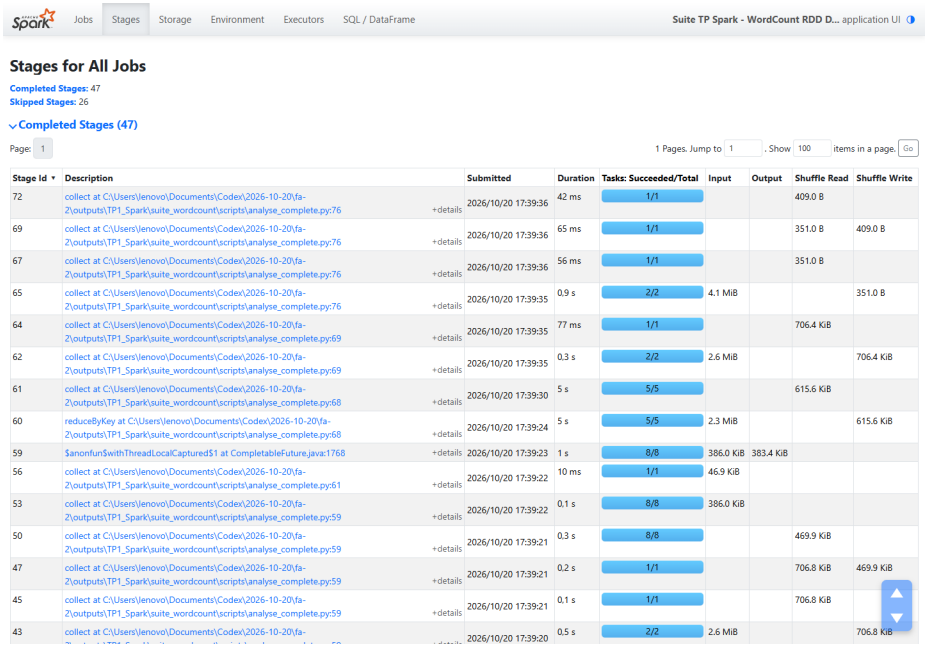



*Figure 14 - Stages de l'application WordCount, capture directe.*

Relevé REST de cette exécution : 43 jobs réussis, 0 jobs en échec, 47 stages terminés et 0 stages en échec. Le contrôle volontaire d'un fichier absent peut échouer avant même la création d'un job. Les nombres sont propres à cette session et ne sont pas des constantes de WordCount.

# 13. DataFrame, SQL et exports

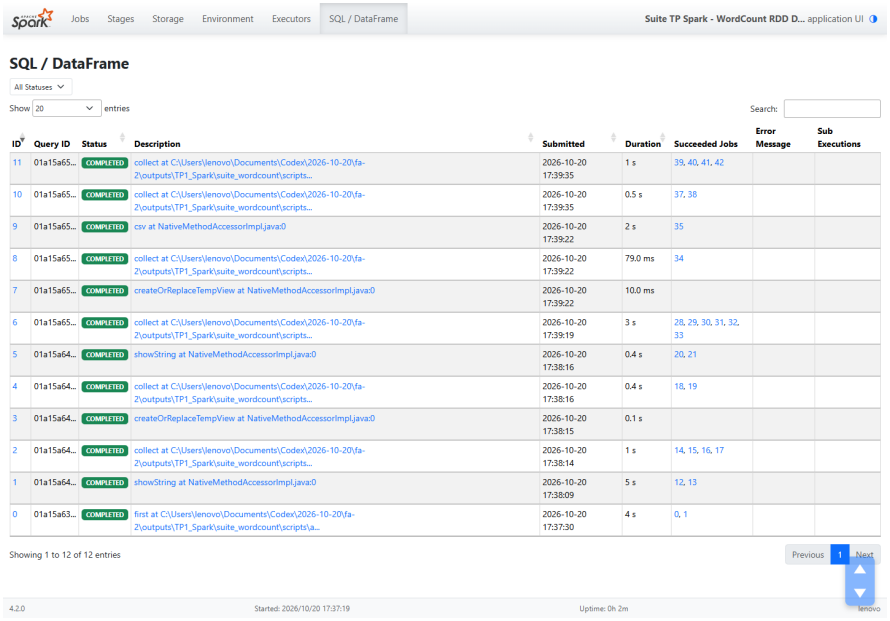



*Figure 15 - Requêtes DataFrame et SQL de la suite.*

Le SQL exprime le résultat attendu, tandis que le pipeline RDD précise les transformations et emploie des fonctions Python dans les workers. Catalyst organise le plan DataFrame. Les différences d'exécution ne justifient pas ici une conclusion de vitesse : aucun benchmark contrôlé n'a été effectué.

## Résultats écrits sur disque

| Dossier / fichier | Contenu |
|---|---|
| resultats/wordcount_rdd/ | Paires (mot, fréquence), fichiers part-* et _SUCCESS |
| resultats/wordcount_df/ | CSV UTF-8 avec en-têtes mot,count, fichiers part-* et _SUCCESS |
| resultats/top15.csv | Classement du corpus brut |
| resultats/top15_sans_vides.csv | Classement après exclusion |
| resultats/par_livre.csv | Comptes par fichier, mots de plus de deux lettres |
| resultats/mesures.json | Mesures, égalités et contrôles complets |

Les exports complets sont relus et comparés aux résultats calculés en mémoire. Une référence indépendante avec collections.Counter confirme toutes les fréquences. Les scripts adaptés refusent un dossier de sortie existant afin de préserver les résultats antérieurs ; pour refaire un essai, choisir un nouveau dossier de sortie.

# 14. Validation et reproduction




*Figure 16 - Contrôles des calculs et des exports.*

## Reproduire uniquement cette suite

Depuis suite_wordcount, utiliser scripts/session_suite.ps1 pour les variables de processus. Le JDK, Python et Spark du projet existant sont réutilisés ; le support Hadoop fourni dans outils est ajouté au PATH. Pour un autre poste, passer les chemins locaux en paramètres. Le script analyse_complete.py contient les nouvelles activités ; wordcount_rdd.py et wordcount_df.py sont les versions autonomes adaptées des scripts fournis.

La première exécution reste archivée. Les exports ne sont pas écrasés ; la procédure de reproduction crée une nouvelle copie de travail horodatée de la suite. Le signal ARRETER_UI termine proprement la session après les captures. Les binaires lourds Java et Spark ne sont pas recopiés dans le ZIP.

## Sources

Nouveau sujet : TP0_TP1_Spark_4.2.0-1.pdf, exercices TP-0, sections 1.1 à 1.6 et annexes. Sources fournies : TP_Spark_sources_et_scripts.zip. Consultation des références le 20 octobre 2026.

*https://spark.apache.org/docs/latest/rdd-programming-guide.html*

*https://spark.apache.org/docs/latest/sql-programming-guide.html*

*https://spark.apache.org/docs/latest/web-ui.html*

*https://github.com/cdarlint/winutils*

*https://www.gutenberg.org/ebooks/17489*

*https://www.gutenberg.org/ebooks/17989*

*https://www.gutenberg.org/ebooks/13951*

*https://www.gutenberg.org/ebooks/5097*

*https://www.gutenberg.org/ebooks/4791*